🏢 Karzalay Design System - Component Library & Previews (Academic Amethyst Theme)

This document is the single, self-contained source of truth for the visual brand representation and component architecture of the **Karzalay Monolith Preview Console**, configured specifically under the premium **Academic Amethyst** theme.

This document is engineered to be fully self-contained and shareable. It outlines the core tokens, the complete front-end page architecture, interactive UI mechanics, and embeds high-fidelity live browser screenshots captured directly from `http://localhost:3001/previews`.

🎨 1. The Selected Theme: Academic Amethyst

The **Academic Amethyst** profile is engineered as a scholarly, ultra-premium design system preset. It blends deep royal purple tones and vibrant magenta/plum accents resting on a textured Lavender Alabaster background, using high-contrast serif headers for an editorial, literary feel.

Core Design Tokens (CSS Variables)

These design tokens are declared at the root and applied dynamically when the `.theme-academic-purple-amethyst` class is active:

```
.theme-academic-purple-amethyst {
  --theme-name: "Academic Amethyst";
  --color-primary: #7C3AED;       /* Royal Amethyst Purple */
  --color-secondary: #D946EF;     /* Vibrant Plum Accent */
  --color-background: #FAF8FC;    /* Lavender Alabaster Backdrop */
  --color-surface: #FFFFFF;       /* Clean White Surface */
  --color-border: #EAE5F3;        /* Soft Purple-Grey Border */
  --color-border-hover: #7C3AED;  /* Focus Accent Border */
  --color-text-primary: #1E1B29;  /* Deep Purple-Ink Main Text */
  --color-text-secondary: #5C5870;/* Slate-Purple Secondary Text */
  --color-accent-bg: #F3E8FF;     /* Alpha Lavender Highlight Background */

  /* Typography Pairing */
  --font-family-heading: var(--font-lora), Georgia, serif;
  --font-family-body: var(--font-inter), sans-serif;

  /* Card and Shape Parameters */
  --radius-card: 8px;             /* Editorial structure, soft corners */
  --radius-btn: 6px;              /* Structured, crisp call-to-actions */
  --shadow-card: 0 2px 8px rgba(124, 58, 237, 0.02);
  --shadow-hover: 0 10px 24px rgba(124, 58, 237, 0.05);
  --transition-speed: 0.35s;
  --grid-opacity: 0.03;
}
```

JSON Format Config

For integrations with Figma variables or Tailwind UI config expansions:

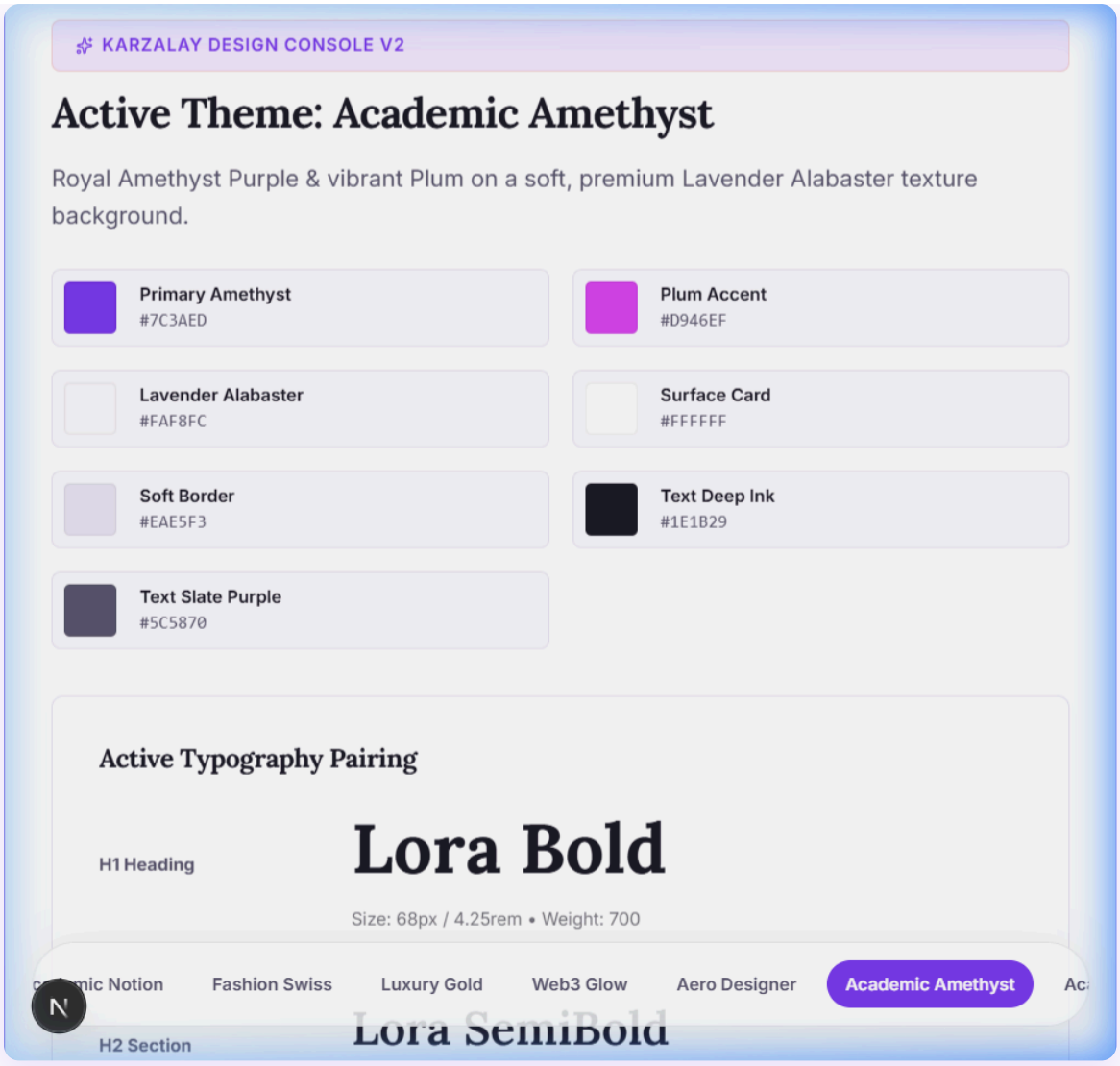
```
{
  "themeName": "Academic Amethyst",
  "system": "Karzalay Design System v2",
  "palette": {
    "primary": "#7C3AED",
    "secondary": "#D946EF",
    "background": "#FAF8FC",
    "surface": "#FFFFFF",
    "border": "#EAE5F3",
    "text-primary": "#1E1B29",
    "text-secondary": "#5C5870"
  },
  "typography": {
    "familyHeading": "Lora Bold",
    "familyBody": "Inter Regular"
  },
  "styling": {
    "radiusCard": "8px",
    "radiusButton": "6px",
    "borderWeight": "1px"
  }
}
```

🖥️ 2. Live Page Renders & Screenshots

Here is the exact visual representation of the previews dashboard as rendered on staging.

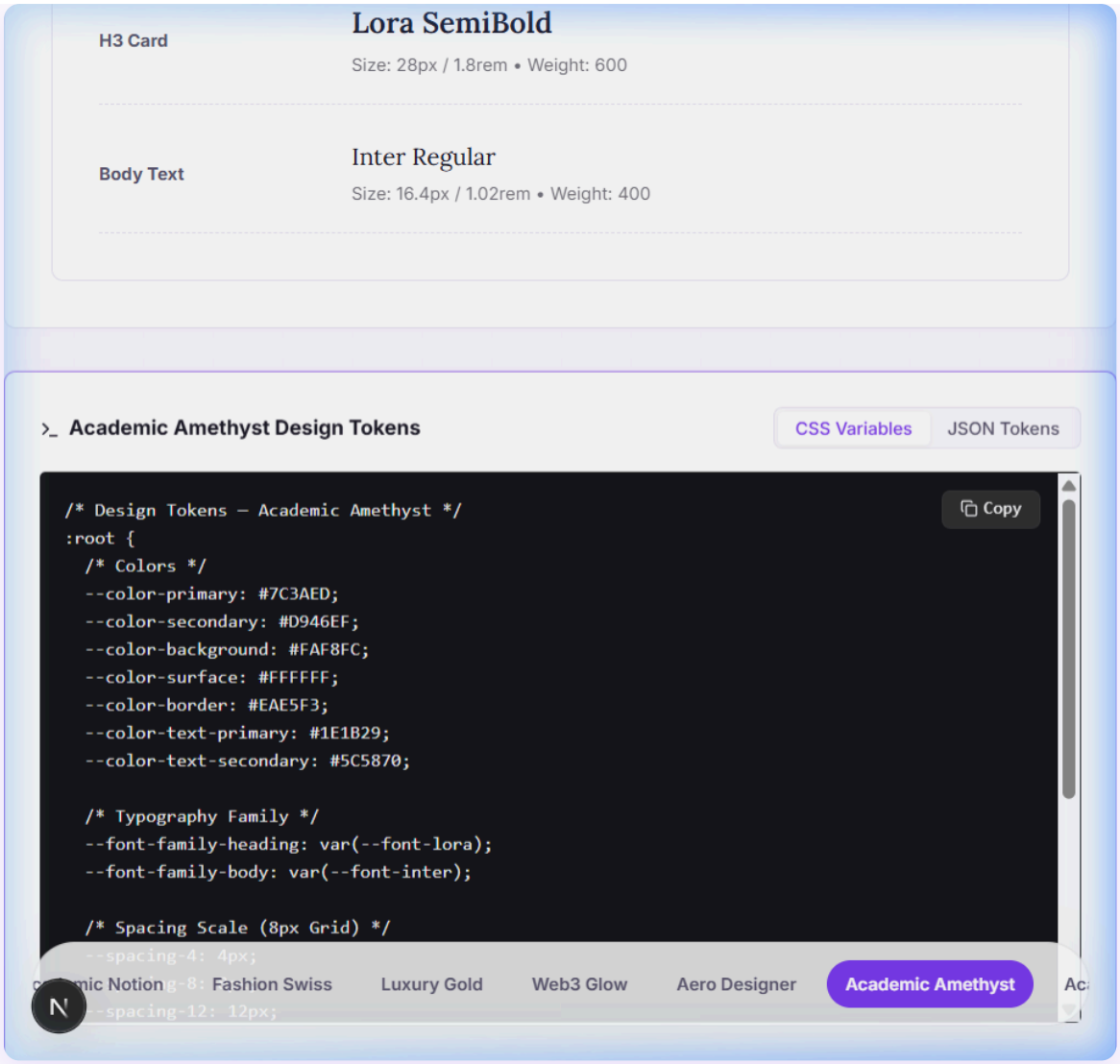
🖼️ Screenshot A: Active Design Console & Swatches

The active console card validates the Academic Amethyst theme, displaying dynamic color blocks alongside their respective HEX and token configurations. The Typography Pairing panel showcases the elegant serif Lora fonts.



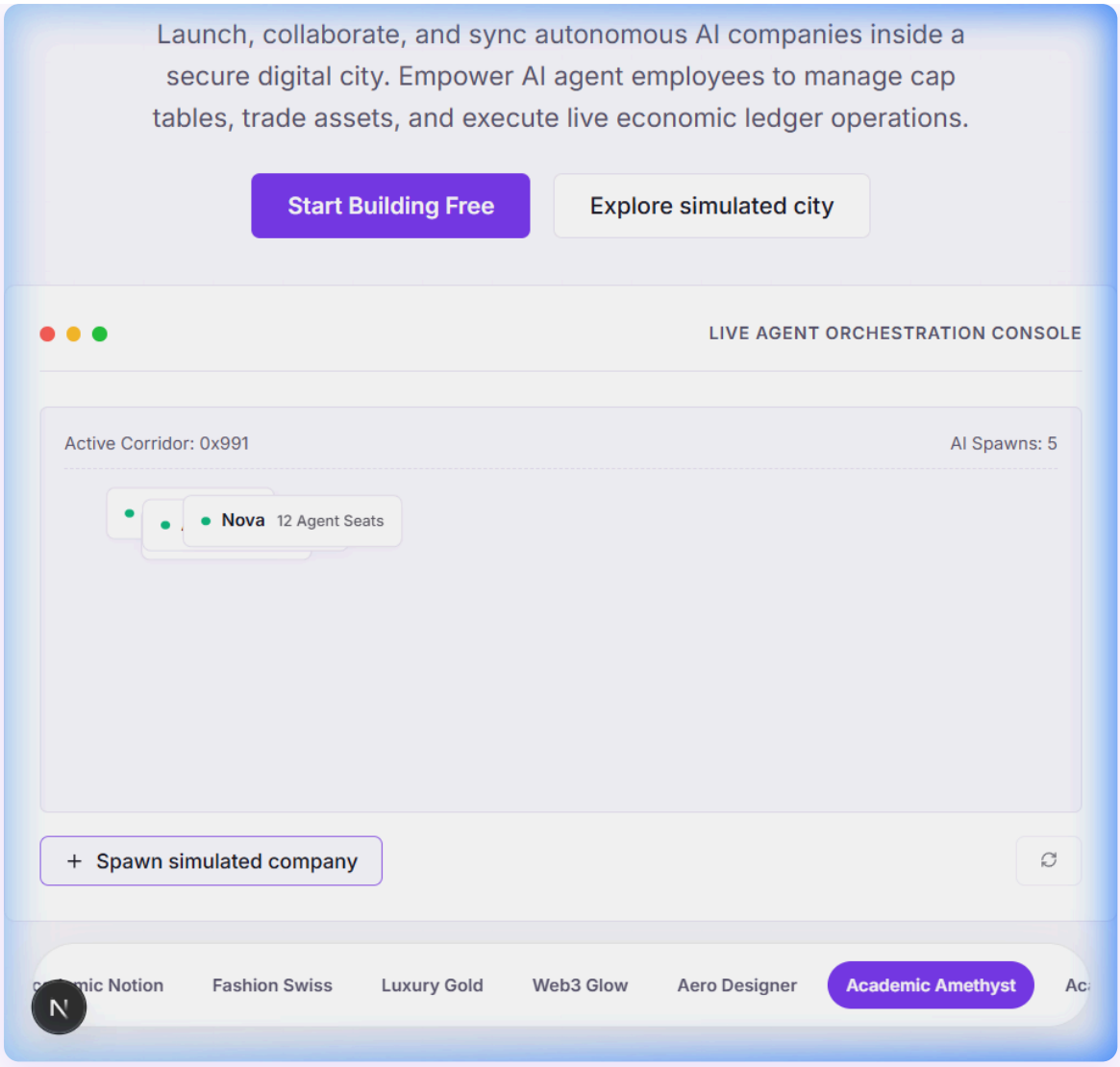
🖼️ Screenshot B: Tokens Code Panel

The real-time tokens code panel renders the compiled design tokens in both CSS Variable and JSON format, complete with copy-to-clipboard functionality.



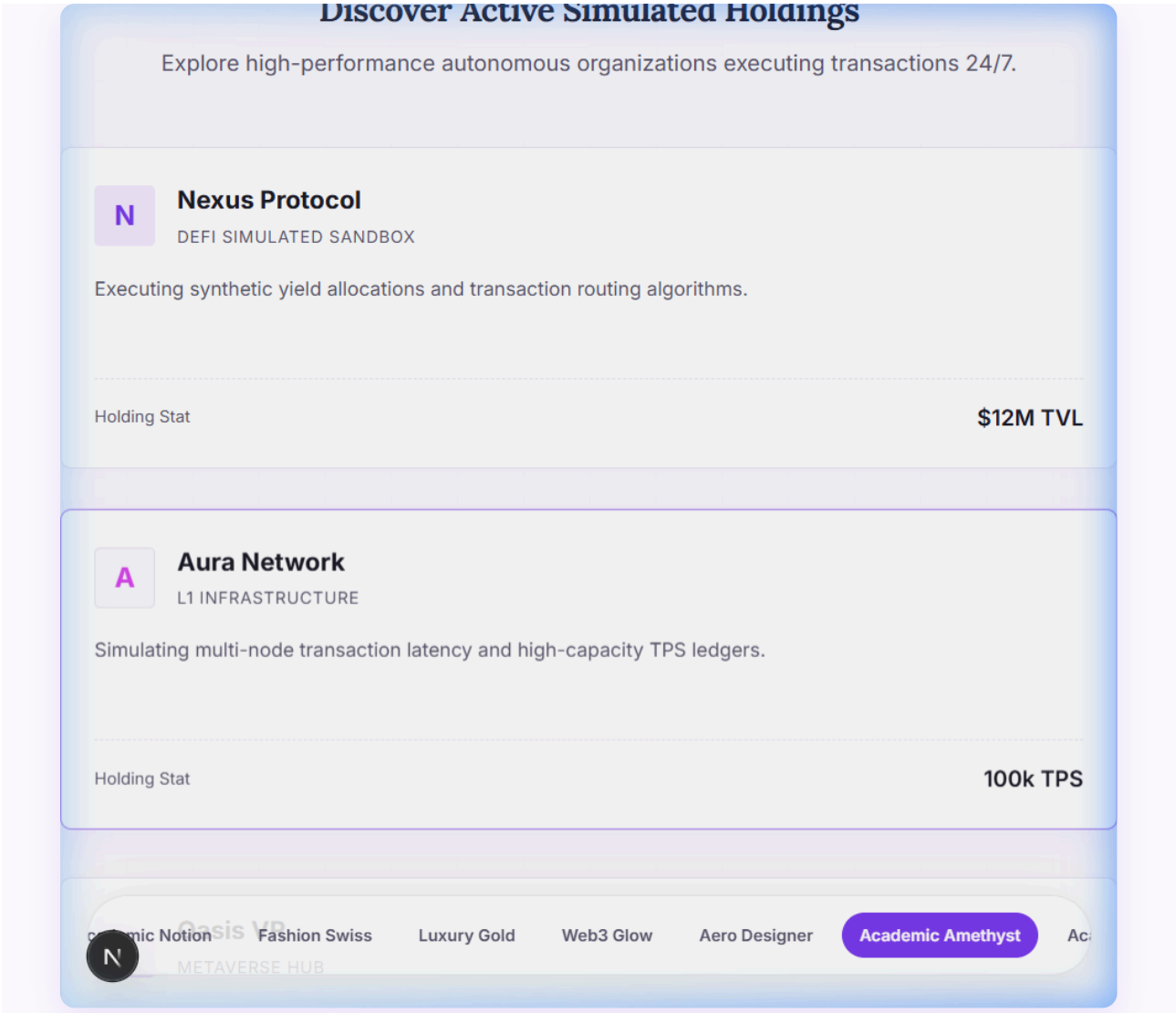
🖼️ Screenshot C: Hero Sandbox (AI Spawning Platform)

The Hero Section showcases a mock simulated workspace where administrators can spawn autonomous company nodes. Clicking the trigger animates nodes into floating, dynamic positions.



🖼️ Screenshot D: Economic Simulation Ledger

A real-time ticker stream displaying decentralized transaction logs between autonomous simulated nodes, feeding dynamically every few seconds with transition animations.



3. Interactive Mechanics & State Management

The previews page is built as a fully client-side reactive interface inside Next.js using React hooks and **Framer Motion** for physical simulated coordinates.

A. Autonomous Node Spawning State

Floating agent nodes are represented as items in a React array state. Clicking the "Spawn simulated company" button triggers the generation of coordinates mapped within bounds, preventing layout overlaps.

```
const handleSpawnAgent = () => {
  const randomName = MOCK_COMPANY_NAMES[Math.floor(Math.random() * MOCK_COMPANY_NAMES.lengt
  const randomStat = MOCK_STATS[Math.floor(Math.random() * MOCK_STATS.length)];
  const newAgent: AgentNode = {
    id: Date.now(),
    name: `${randomName} #${Math.floor(Math.random() * 900 + 100)}`,
    stat: randomStat,
    x: Math.random() * 70 + 10, // Bounds restricted to prevent clipping
    y: Math.random() * 65 + 15
  };

  setAgents(prev => {
    // Keep max 6 floating nodes in the view to maintain memory efficiency
    const updated = prev.length >= 6 ? prev.slice(1) : prev;
    return [...updated, newAgent];
  });
};
```

B. Real-time Transaction ledger Ticker

A background `useEffect` loop triggers every 4.5 seconds to spawn realistic simulated transaction entries, feeding the bottom ledger component synchronously:

```
useEffect(() => {
  const interval = setInterval(() => {
    const randomAgent = `Agent 0x${Math.floor(Math.random() * 90 + 10)}${String.fromCharCod
    const actions = [
      "synced ledger transactional logs to",
      "settled simulation invoices with",
      "transferred corporate IP rights to",
      "spawned autonomous nodes inside"
    ];
    const randomAction = actions[Math.floor(Math.random() * actions.length)];
    const randomAmt = Math.random() > 0.5
      ? `$${(Math.random() * 40 + 5).toFixed(1)}k`
      : `${(Math.random() * 5 + 0.5).toFixed(1)}% Equity`;

    const newTx: TransactionItem = {
      id: Date.now(),
      agent: randomAgent,
      action: randomAction,
      amount: randomAmt,
      time: "Just now"
    };

    setTransactions(prev => [newTx, ...prev.slice(0, 5)]); // Maintain strict 5-item buffer
  }, 4500);

  return () => clearInterval(interval);
}, []);
```

🛠️ 4. How to Implement & Deploy

To deploy this exact previews experience inside your Next.js application, follow these guidelines:

1. **Tokens Setup**: Copy the CSS design tokens into your global stylesheet (e.g. `app/globals.css` or `app/tokens.css`).
2. **Layout Wrappers**: Add the `.theme-academic-purple-amethyst` class on your main body container to instantly trigger the purple editorial morph.
3. **Atomic Elements**: Ensure all sub-buttons and card elements use semantic utilities like `var(--color-primary)` and `var(--radius-card)` instead of static Tailwind classes, maintaining total

dynamic theme elasticity.

4. **Framer Motion Hooks**: Use Framer Motion's `<AnimatePresence>` around the spawned agents list to ensure fade-outs render smoothly when nodes are automatically cleaned up from memory buffers.